

*P2025 : Outil de surveillance de l'état du terrain d'un
hippodrome*

Maillot Nathan

Dossier technique du projet - partie individuelle

Table des matières

1 -SITUATION DANS LE PROJET	2
1.1 -SYNOPTIQUE DE LA REALISATION	2
1.2 -DESCRIPTION DE LA PARTIE PERSONNELLE.....	2
2 -REALISATION DU CAS D'UTILISATION TRANSMETTRE LES VALEURS ACQUISES	2
2.1 -CONCEPTION DETAILLEE.....	3
2.2 -TESTS UNITAIRES	4
2.2.1 -Test unitaire du module matériel/logiciel xxx.....	4
2.2.2 -Problèmes rencontrés	5
3 -REALISATION DU CAS D'UTILISATION ARCHIVER LES DONNES	5
3.1 -CONCEPTION DETAILLEE.....	6
3.2 -TESTS UNITAIRES	8
3.3.1 -Test unitaire du module matériel/logiciel xxx.....	9
3.3.2 -Problèmes rencontrés	9
4 -REALISATION DU CAS D'UTILISATION RECEVOIR LES DONNEES	9
4.1 -CONCEPTION DETAILLEE.....	10
4.2 -TESTS UNITAIRES	15
4.2.1 -Test unitaire du module matériel/logiciel xxx.....	15
4.2.2 -Problèmes rencontrés	17
5 -BILAN DE LA REALISATION PERSONNELLE	17
6 -ANNEXE	18

1 - Situation dans le projet

1.1 - Synoptique de la réalisation

Le projet consiste à mettre en place une chaîne complète de surveillance environnementale d'un hippodrome, basée sur des capteurs IoT LoRaWAN, une infrastructure logicielle pour la réception et le traitement des données, et une interface d'accès via une API REST. L'objectif est de permettre un suivi en temps réel de l'état du terrain, des conditions météo et des actions d'arrosage, afin de garantir la sécurité des chevaux et la qualité des courses.

Le schéma synoptique de la réalisation est organisé comme suit :

- Capteurs LoRaWAN (température, humidité, vent, etc.) installés sur différentes zones de la piste.
- Une passerelle LoRaWAN qui transmet les trames des capteurs au serveur.
- Le réseau ChirpStack v4, qui reçoit les données via le service Gateway Bridge et les rend disponibles sous forme de paquets décodés.
- Une base de données PostgreSQL, utilisée pour stocker les mesures météo, les données terrain et les métadonnées des capteurs.
- Une API REST développée en NestJS, qui expose les données pour une exploitation externe (interface graphique, analyse, etc.).
- Une interface IHM en C pour afficher les données transmises par l'API

1.2 - Description de la partie personnelle

Dans ce projet, mon rôle était de gérer la réception, le stockage et la diffusion des données envoyées par la passerelle LoRaWAN. Concrètement, cela voulait dire configurer le serveur ChirpStack pour recevoir les données, les stocker dans une base PostgreSQL et développer une API REST pour que ces données puissent être accessibles. J'étais responsable de tout ce processus, depuis la réception des données jusqu'à leur diffusion via l'API.

Une fois l'environnement de travail de test installé avec Ubuntu 22.04, PostgreSQL, ChirpStack et NestJS, j'ai commencé à configurer ChirpStack pour recevoir les trames envoyées par la passerelle. Cela a impliqué la création des devices dans ChirpStack et l'intégration avec MQTT pour assurer la réception des données. En même temps, j'ai conçu la base de données avec des tables pour les données météorologiques, les mesures de terrain et les capteurs, afin de les organiser correctement.

Ensuite, j'ai mis en place un système qui insère automatiquement les données reçues dans la base de données. Ces données étaient envoyées au format JSON par ChirpStack, et j'ai créé un processus pour les convertir et les stocker correctement dans PostgreSQL. Après cela, j'ai développé l'API REST avec NestJS pour créer des routes permettant d'accéder aux données stockées dans la base.

2 - Réalisation du cas d'utilisation transmettre les valeurs acquises

Dans cette section, nous détaillons la réalisation de la tâche professionnelle relative à la réception des données depuis la passerelle LoRaWAN via ChirpStack.

La réception des données a été une étape clé du projet, et elle a été gérée principalement par le serveur ChirpStack. Voici comment cette tâche a été conçue et mise en œuvre :

Choix technologiques :

Chirpstack	ChirpStack est une solution open-source qui permet de gérer un réseau LoRaWAN. Ce logiciel donne un contrôle total sur l'infrastructure et les paramètres du réseau, ce qui le rend idéal pour des projets où la personnalisation est importante. Un grand avantage de ChirpStack est qu'il permet d'adapter facilement le système aux besoins
------------	--

	spécifiques du projet, tout en offrant une gestion complète des dispositifs et des passerelles.
The Things Network (TTN)	The Things Network (TTN) est une plateforme IoT publique et gratuite qui simplifie le déploiement de réseaux LoRaWAN. Il est facile à mettre en place et offre une couverture large, idéale pour les projets de petite et moyenne envergure. Un avantage de TTN est qu'il permet de se concentrer sur le développement des applications IoT, sans se soucier de la gestion des infrastructures réseau.

J'ai choisi ChirpStack v4 pour gérer la réception des trames LoRaWAN. ChirpStack est un serveur open-source spécialisé dans la gestion des réseaux LoRaWAN, qui permet une intégration facile avec des passerelles LoRaWAN et la réception des données en temps réel.

UDP	UDP est un protocole de communication qui permet de transmettre des données rapidement et avec une faible latence. Il est particulièrement adapté pour les applications où la vitesse de transmission est cruciale, même si certaines pertes de données peuvent être acceptées. L'avantage principal de UDP est sa rapidité, car il ne prend pas de temps à établir une connexion ou à vérifier la réception des données.
TCP	TCP est un protocole de communication qui assure la fiabilité des transmissions de données. Il garantit que les données arrivent dans le bon ordre et sans erreur, ce qui est essentiel pour des applications critiques où la perte de données n'est pas acceptable. Le principal avantage de TCP est sa fiabilité, car il assure une transmission sécurisée, en vérifiant que chaque donnée est reçue correctement avant de continuer.

Le choix du protocole UDP pour la communication entre la passerelle et le serveur ChirpStack s'est imposé car il permet une transmission rapide des données avec une faible latence. Bien que le protocole UDP ne garantisse pas la fiabilité des paquets, il est parfaitement adapté aux transmissions de données fréquentes et légères, comme celles envoyées par la passerelle LoRaWAN.

2.1 - Conception détaillée

La réception des données a été une étape clé du projet, et elle a été gérée principalement par le serveur ChirpStack. Voici comment cette tâche a été conçue et mise en œuvre :

Installer les paquets nécessaires pour cette partie	<pre>sudo apt install mosquito sudo apt install mosquito-clients sudo apt install redis-server sudo apt install redis-tools sudo apt install postgresql</pre>
Se connecter à PostgreSQL	<pre>sudo -u postgres psql</pre>
Créer un utilisateur pour ChirpStack	<pre>create role chirpstack with login password 'chirpstack';</pre>
Créer la base de données ChirpStack et l'associer à l'utilisateur créé	<pre>create database chirpstack with owner chirpstack;</pre>

Se connecter à la base de données ChirpStack	\c chirpstack
Créer l'extension nécessaire pour le traitement des données	create extension pg_trgm;
Quitter PostgreSQL	\q
Installer un référentiel de logiciels	sudo apt install apt-transport-https dirmngr
Ajouter la clé GPG du référentiel ChirpStack	sudo apt-key adv --keyserver keyserver.ubuntu.com --recv-keys 1CE2AFD36DBCCA00
Ajouter le référentiel ChirpStack à la liste des sources	sudo echo "deb https://artifacts.chirpstack.io/packages/4.x/deb stable main" sudo tee /etc/apt/sources.list.d/chirpstack.list
Mise à jour du cache	sudo apt update
Installation de Chirpstack Gateway Bridge	sudo apt install chirpstack-gateway-bridge
Configuration en EU868 dans /etc/chirpstack-gateway-bridge/chirpstack-gateway-bridge.toml	[integration.mqtt] event_topic_template="eu868/gateway/{{ .GatewayID }}/event/{{ .EventType }}" command_topic_template="eu868/gateway/{{ .GatewayID }}/command/#"
Configuration du serveur d'adresse dans /etc/chirpstack-gateway-bridge/chirpstack-gateway-bridge.toml[integration.mqtt]	[backend] Udp_bind=192.168.20.30 :1700
Démarrage de Chirpstack Gateway Bridge	sudo systemctl start chirpstack-gateway-bridge
Démarrer le service au démarrage de l'appareil	sudo systemctl enable chirpstack-gateway-bridge
Installation de Chirpstack	apt install chirpstack
Configuration de l'adresse url dans /etc/chirpstack/chirpstack.toml	[api] Bind=192.168.20.3 :8080
Démarrer Chirpstack	sudo systemctl start chirpstack
Démarrer Chirpstack au démarrage	sudo systemctl enable chirpstack
Afficher le journal de Chirpstack	sudo journalctl -f -n 100 -u chirpstack

Cette configuration permet à ChirpStack de recevoir les données des capteurs via la passerelle LoRaWAN.

2.2 - Tests unitaires

Après avoir terminé l'installation et la configuration de ChirpStack, plusieurs tests unitaires ont été réalisés afin de s'assurer du bon fonctionnement de la communication entre la passerelle LoRaWAN, ChirpStack, et la base de données. Les tests ont été effectués progressivement, en commençant par la vérification des services installés, puis en simulant l'envoi de données depuis une passerelle, et enfin en vérifiant leur stockage et accessibilité.

2.2.1 - Test unitaire du module logiciel : Réception d'une trame LoRaWAN

Élément testé :	Chirpstack – Réception de données via la passerelle		
Objectif du test :	Vérifier que Chirpstack reçoit bien une trame LoRaWAN transmise par la passerelle		
Nom du testeur :	Nathan MAILLOT	Date :	30/04/2025
Moyens mis en œuvre :	Logiciel : ChirpStack v4, journal système (journalctl)	Matériel : Passerelle LoRaWAN configurée (IP : 192.168.20.9, port : 1700)	Outil de développement : Terminal Ubuntu, éditeur de

			configuration, outils réseau (ping, netcat)	
Procédure du test :				
Id	Description du vecteur de test	Résultat attendu	Résultat obtenu	Validation (O/N)
1	Démarrage du service chirpstack-gateway-bridge	Service actif, écoute sur port UDP 1700	Démarrage sans problème	O
2	Démarrage du service chirpstack	Service actif	Affichage de l'interface avec l'adresse « 192.168.20.3 :8080 »	O
3	Connexion de la passerelle sur le service	Apparition de la passerelle en mode online sur le site	Passerelle en mode online avec voyant vert	O
4	Réception de données sur l'interface Chirpstack	Réception de trames sur LoRaWAN frames	unconfirmedDataUp avec la valeur « hello »	O
Conclusion du test :		L'ensemble des services liés à ChirpStack ont été démarrés correctement, la passerelle s'est bien connectée, et les trames LoRaWAN ont été reçues comme attendu. Cela confirme le bon fonctionnement de l'intégration entre la passerelle et le serveur ChirpStack.		

2.2.2 - Problèmes rencontrés

Problème1 : La passerelle ne s'affichait pas en ligne dans l'interface

Solution : Une erreur de configuration de l'adresse IP du serveur dans le fichier chirpstack-gateway-bridge.toml a été corrigée.

Problème2 : Aucune trame n'était reçue dans l'interface

Solution : Création d'un nouveau device pour avoir les infos importantes

3 - Réalisation du cas d'utilisation archiver les données

Dans ce projet, le stockage des données constitue une étape essentielle, car il permet de conserver toutes les informations envoyées par les capteurs via la passerelle. Ces données doivent être stockées de manière fiable, structurée et évolutive, afin de pouvoir être consultées plus tard par l'API ou toute autre application métier.

Avant de choisir un outil de stockage adapté au projet, il a été nécessaire de comparer les deux grandes familles de bases de données : relationnelles et non-relationnelles.

Choix technologiques :

Base de données relationnelle	Ces bases utilisent des tables avec des colonnes et des lignes. Elles sont très bien adaptées lorsque les données sont bien structurées et que les relations entre elles sont importantes. On peut y faire des recherches précises et complexes avec le langage SQL. Elles sont très utilisées dans les projets professionnels.
Base de données non-relationnelle (NoSQL)	Ces bases sont plus souples. Elles permettent de stocker des données sous forme de documents, de listes ou de simples paires clé/valeur. Elles sont souvent utilisées quand les données changent souvent de structure ou quand la vitesse de traitement est prioritaire. Moins adaptées aux projets où la cohérence entre les données est essentielle.

Dans le cadre de ce projet, le besoin était de stocker des données provenant de capteurs avec une structure connue comme valeurs, dates, zones et encore plus, tout en gardant une certaine souplesse pour faire évoluer les types de mesures. Une base relationnelle s'est vite imposée, car elle permettait une meilleure maîtrise des formats de données, une interopérabilité avec l'API à développer.

Une fois le type de base de données choisi — en l'occurrence une base — il a été nécessaire de se pencher sur les différentes applications capables de gérer ce type de base. L'objectif était de trouver un système fiable, compatible avec l'environnement du projet, facile à mettre en place, et qui dispose d'une bonne documentation pour faciliter l'intégration et la maintenance. Cette étape a permis de comparer plusieurs solutions afin d'en retenir une qui réponde au mieux aux besoins techniques et pratiques du projet.

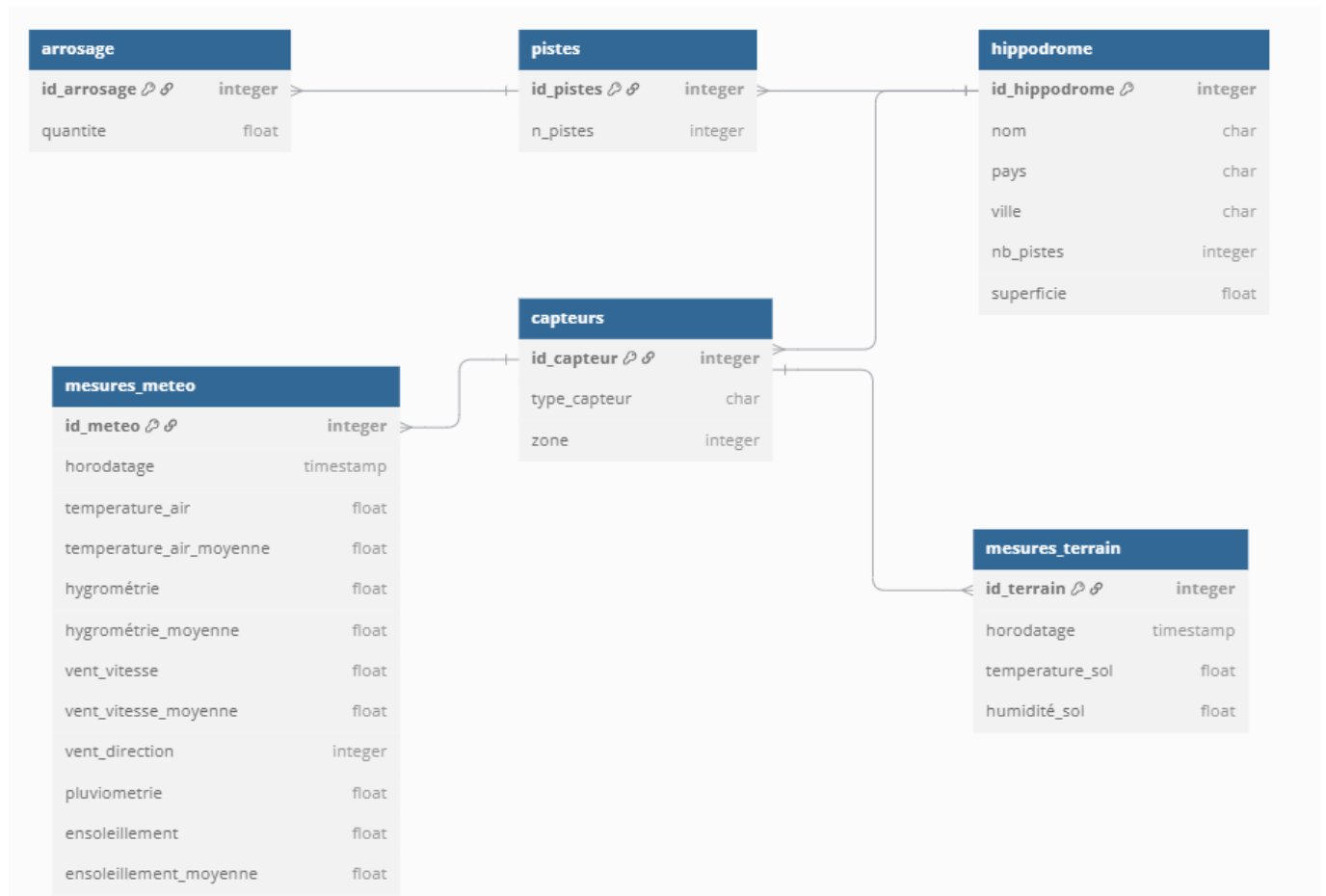
Choix technologiques :

PostgreSQL	Système de gestion de base de données relationnelle open source, très robuste et performant. Il offre une bonne compatibilité avec les types de données avancés comme JSONB, ce qui permet de stocker des données flexibles sans sacrifier la structure. PostgreSQL est bien documenté, largement utilisé dans l'industrie
MySQL	Autre SGBD relationnelle open source, aussi très utilisé. Cependant, il propose une gestion des données JSON moins avancée, ce qui limite sa flexibilité dans le cadre de ce projet où les données des capteurs peuvent varier d'un type à un autre.
MongoDB	Base de données orientée documents, idéale pour les données non structurées ou très variables. Elle aurait pu être utilisée pour sa grande souplesse, mais son intégration avec le reste de la chaîne technique (notamment l'API REST et les outils existants en place) aurait demandé des ajustements plus importants.
SQLite	Légère et facile à mettre en place, cette base de données locale aurait été envisageable pour des tests ou un prototype. Toutefois, elle n'est pas adaptée à un fonctionnement en production ou en environnement multi-utilisateurs avec de gros volumes de données.

Après avoir comparé plusieurs systèmes de gestion de bases de données relationnelles, le choix s'est porté sur PostgreSQL. Ce système est reconnu pour sa fiabilité, sa robustesse et sa conformité aux standards SQL. Il est également open source, ce qui facilite son installation et son utilisation sans coût supplémentaire. PostgreSQL offre de bonnes performances, une large communauté active, et une compatibilité parfaite avec les outils utilisés dans le reste du projet, notamment pour le stockage structuré et sécurisé des données issues des capteurs.

3.1 - Conception détaillée

Afin de permettre le stockage structuré et efficace des données reçues des capteurs, j'ai conçu une base de données relationnelle en suivant le schéma défini lors de l'analyse. Ce schéma est adapté aux types de mesures collectées sur l'hippodrome, que ce soient des données météorologiques, de terrain ou liées à l'arrosage. La conception des tables permet de relier chaque mesure à son capteur et de différencier les types de données pour faciliter leur traitement et leur visualisation. Voici les tables que j'ai créées à partir de ce schéma :



Installer PostgreSQL	sudo apt install postgresql
Accéder à l'utilisateur postgres	sudo -u postgres psql
Création de la table Hippodrome	<pre>CREATE TABLE hippodrome (id_hippodrome SERIAL PRIMARY KEY, nom CHAR(100), pays CHAR(50), ville CHAR(50), nb_pistes INT, superficie FLOAT);</pre>
Création de la table Capteur	<pre>CREATE TABLE capteurs (id_capteur SERIAL PRIMARY KEY, id_hippodrome INTEGER REFERENCES hippodrome(id_hippodrome), type_capteur CHAR(50), zone INT);</pre>
Création de la table Mesures_meteo	<pre>CREATE TABLE mesures_meteo (id_meteo SERIAL PRIMARY KEY, id_capteur INTEGER REFERENCES capteurs(id_capteur), horodatage TIMESTAMP, temperature_air FLOAT, temperature_air_moyenne FLOAT, hygrometrie FLOAT, hygrometrie_moyenne FLOAT, vent_vitesse FLOAT, vent_vitesse_moyenne FLOAT, vent_direction INTEGER, pluviometrie FLOAT, ensoleillement FLOAT, ensoleillement_moyenne FLOAT);</pre>

	<pre> id_hippodrome INTEGER REFERENCES hippodrome(id_hippodrome), type_capteur CHAR(50), zone INT); </pre>
Création de la table Mesures_terrain	<pre> CREATE TABLE mesures_terrain (id_terrain SERIAL PRIMARY KEY, id_capteur INTEGER REFERENCES capteurs(id_capteur), horodatage TIMESTAMP, temperature_sol FLOAT, humidité_sol FLOAT); </pre>
Création de la table Arrosage	<pre> CREATE TABLE arrosage (id_arrosage SERIAL PRIMARY KEY, quantite FLOAT); </pre>
Création de la table pistes	<pre> CREATE TABLE pistes (id_pistes SERIAL PRIMARY KEY, id_hippodrome INTEGER REFERENCES hippodrome(id_hippodrome), id_arrosage INTEGER REFERENCES arrosage(id_arrosage), n_pistes INT); </pre>

La base de données est désormais prête à accueillir les données issues des capteurs, en assurant une organisation claire et évolutive, indispensable pour les étapes suivantes du projet.

3.2 - Tests unitaires

Pour valider le bon fonctionnement du stockage des données dans la base PostgreSQL, plusieurs tests unitaires ont été réalisés de manière chronologique. Ces tests ont permis de vérifier la cohérence du modèle relationnel, la conformité des types de données, l'intégrité des relations entre les tables, ainsi que la capacité à effectuer des opérations d'insertion, de sélection ou encore de suppression.

Les premiers tests ont concerné la création des tables et des relations. Ensuite, des données de test ont été insérées manuellement pour vérifier la validité du schéma. Enfin, des requêtes SQL simulant un cas d'utilisation réel (par exemple, l'enregistrement d'une mesure météo associée à un capteur) ont été exécutées.

Parmi ces tests, celui portant sur l'insertion et la liaison d'une mesure météo à un capteur via l'ID a été sélectionné pour être détaillé ci-dessous.

3.2.1 - Test unitaire du module base de données : Insertion d'une mesure météo

Élément testé :	Table Mesures_meteo et sa relation avec capteurs		
Objectif du test :	Vérifier que l'insertion d'une mesure météo liée à un capteur fonctionne sans erreur et respecte les contraintes du schéma		
Nom du testeur :	Nathan MAILLOT	Date :	30/04/2025
Moyens mis en œuvre :	Logiciel : PostgreSQL (via PgAdmin)	Matériel : Machine virtuelle Ubuntu 22.04	Outil de développement : Éditeur SQL intégré (psql)

Procédure du test :				
Id	Description du vecteur de test	Résultat attendu	Résultat obtenu	Validation (O/N)
1	Insertion d'un capteur de type météo	Capteur ajouté sans erreur	Capteur ajouté	O
2	Insertion d'une mesure météo liée au capteur	Mesure insérée avec clé étrangère valide	Mesure ajouté	O
3	Insertion d'une mesure météo avec un capteur inexistant	Erreur de contrainte étrangère	ERROR: insert or update on table "mesures_terrain" violates foreign key constraint "FK_1fcda743a84013cb18ce005d9e5" Key (id_capteur)=(3) is not present in table "capteur".	O
4	Insertion avec un champ de type incorrect (ex : string au lieu de float)	Erreur de type	Valeur ajouté	N
5	Sélection des mesures jointes aux capteurs	Affichage correct des valeurs	Renvoi l'affichage des valeurs associés à l'id du capteur	O
Conclusion du test :		Les différents cas d'insertion ont permis de vérifier que la structure relationnelle entre les capteurs et les mesures météo est correctement définie. Les contraintes de types et les clés étrangères jouent leur rôle en bloquant quelques incohérences cependant il faut faire attention au champ string. La base de données est prête à stocker les mesures transmises par les capteurs en respectant les données.		

3.2.2 -Problèmes rencontrés

Problème 1 : Une erreur de contrainte de clé étrangère apparaissait lors d'une insertion de mesure météo sans capteur associé.

Solution : Il est rappelé qu'un capteur doit obligatoirement exister avant d'associer une mesure. Une insertion du capteur a donc été ajoutée.

Problème 2 : PostgreSQL accepte une chaîne de caractères contenant un chiffre dans un champ float sans générer d'erreur

4 - Réalisation du cas d'utilisation recevoir les données

Pour permettre l'accès aux données de la base par une interface externe (comme une IHM ou une application mobile), il est nécessaire de créer une API. Deux APIs existent : les APIs REST (basées sur HTTP) et les APIs GraphQL. Dans ce projet, l'objectif est de proposer une API simple et compatible avec les formats de données attendus

Type d'interface : API REST ou GraphQL

Une API permet à une application d'accéder aux données d'un serveur via un protocole bien défini. Deux approches sont généralement utilisées :

API REST	C'est le type d'API le plus courant. Elle repose sur les méthodes HTTP standards (GET, POST, PUT, DELETE) et permet une structure simple et claire des échanges. Son principal avantage est sa grande compatibilité avec de nombreuses interfaces front-end.
API GraphQL	C'est une approche plus récente qui permet de demander exactement les données nécessaires, dans une seule

	requête. Elle est plus souple côté client, mais plus complexe à mettre en place et à sécuriser.
--	---

Une fois le type d'API défini, plusieurs outils peuvent être utilisés pour la concevoir :

Express.js (Node.js)	Basé sur Node.js, il est simple, rapide à mettre en place, et très documenté. C'est un bon choix pour les petits projets.
Spring Boot (Java)	Framework Java robuste, adapté aux grandes applications. Très utilisé dans le monde professionnel.
Django Rest Framework (Python)	Basé sur Python, il permet une intégration rapide avec des bases relationnelles. Il est très lisible, mais plus adapté à ceux maîtrisant Python.
NestJS (Node.js / TypeScript)	Framework Node.js utilisant TypeScript. Il est structuré, modulaire et inspiré d'Angular, ce qui facilite le travail en équipe. Il offre un bon équilibre entre performance, lisibilité et évolutivité.

Le développement de l'API s'est fait avec le framework NestJS, en mode REST, car il offre une architecture claire, une bonne intégration avec PostgreSQL et une facilité de maintenance en TypeScript.

4.1 - Conception détaillée

Pour interagir avec la base de données PostgreSQL, une API a été développée en utilisant le framework NestJS, basé sur Node.js et TypeScript. Cette API suit une architecture REST pour permettre la consultation et l'enregistrement des données issues des capteurs.

Installation de NestJS	<code>npm install -g @nestjs/cli</code>
Création d'un nouveau projet	<code>nest new projet-api</code>
Déplacement vers le projet	<code>cd projet-api</code>
Installation des dépendances	<code>npm install --save @nestjs/typeorm typeorm pg</code>
Installation de Visual Studio Code	(Annexe 1)
Installation de NestJS Files	(Annexe 2)
Configuration app.module.ts	<pre>import { Module } from '@nestjs/common'; import { TypeOrmModule } from '@nestjs/typeorm'; import { AppController } from './app.controller'; import { AppService } from './app.service'; import { HippodromeModule } from './hippodrome/hippodrome.module'; import { CapteurModule } from './capteur/capteur.module'; import { MesuresMeteoModule } from './mesures_meteo/mesures_meteo.module'; import { MesuresTerrainModule } from './mesures_terrain/mesures_terrain.module'; import { ArrosageModule } from './arrosage/arrosage.module'; import { PistesModule } from './pistes/pistes.module'; import { Hippodrome } from './hippodrome/hippodrome.entity'; import { Capteur } from './capteur/capteur.entity'; import { MesuresMeteo } from './mesures_meteo/mesures_meteo.entity';</pre>

	<pre>import { MesuresTerrain } from './mesures_terrain/mesures_terrain.entity'; import { Pistes } from './pistes/pistes.entity'; import { Arrosage } from './arrosage/arrosage.entity'; @Module({ imports: [TypeOrmModule.forRoot({ type: 'postgres', host: 'localhost', port: 5432, username: "postgres", password: 'password', database: 'hippodrome_db', entities: [Hippodrome, Capteur, MesuresMeteo, MesuresTerrain, Pistes, Arrosage], synchronize: false, }), HippodromeModule, CapteurModule, MesuresMeteoModule, MesuresTerrainModule, ArrosageModule, PistesModule], }) export class AppModule {}</pre>
Création des modules (hippodrome, capteur, mesures-meteo, mesures-terrain)	<pre>nest generate module hippodrome nest generate module capteur nest generate module mesures-meteo nest generate module mesures-terrain nest generate module arrosage nest generate module pistes</pre>
Création des services pour chaque module	<pre>nest generate service hippodrome nest generate service capteur nest generate service mesures-meteo nest generate service mesures-terrain nest generate service arrosage nest generate service pistes</pre>
Création des entités TypeORM	<pre>nest generate class hippodrome/hippodrome.entity nest generate class capteur/capteur.entity nest generate class mesures-meteo/mesures-meteo.entity nest generate class mesures-terrain/mesures-terrain.entity nest generate class arrosage/arrosage.entity nest generate class pistes/pistes.entity</pre>
Création des contrôleurs pour chaque module	<pre>nest generate controller hippodrome nest generate controller capteur nest generate controller mesures-meteo nest generate controller mesures-terrain nest generate controller arrosage nest generate controller pistes</pre>

Intégration de TypeORM dans les modules	<pre>import { Module } from '@nestjs/common'; import { TypeOrmModule } from '@nestjs/typeorm'; import { MesuresMeteo } from './mesures_meteo.entity'; import { MesuresMeteoService } from './mesures_meteo.service'; import { MesuresMeteoController } from './mesures_meteo.controller'; @Module({ imports: [TypeOrmModule.forFeature([MesuresMeteo])], providers: [MesuresMeteoService], controllers: [MesuresMeteoController] }) export class MesuresMeteoModule {}</pre>
Création de l'entité qui contient les éléments de la bases de données	<pre>import { Entity, PrimaryGeneratedColumn, Column, ManyToOne, JoinColumn } from "typeorm"; import { Capteur } from "src/capteur/capteur.entity"; @Entity() export class MesuresMeteo { @PrimaryGeneratedColumn() id_meteo: number; @ManyToOne(() => Capteur, (capteur) => capteur.mesuresMeteo) @JoinColumn({ name : 'id_capteur' }) capteur: Capteur; @Column() horodatage: Date; @Column('float') temperature_air: number; @Column('float') temperature_air_moyenne: number; @Column('float') hygrometrie: number; @Column('float') hygrometrie_moyenne: number; @Column('float') vent_vitesse: number; @Column('float') vent_vitesse_moyenne: number;</pre>

	<pre> @Column('int') vent_direction: number; @Column('float') pluviometrie: number; @Column('float') ensoleillement: number; @Column('float') ensoleillement_moyenne: number; } </pre>
Création du service de mesures_meteo qui renvoie toutes les valeurs	<pre> import { Injectable } from '@nestjs/common'; import { InjectRepository } from '@nestjs/typeorm'; import { Repository } from 'typeorm'; import { MesuresMeteo } from './mesures_meteo.entity'; @Injectable() export class MesuresMeteoService { constructor(@InjectRepository(MesuresMeteo) private mesuresMeteoRepository: Repository<MesuresMeteo>){} async findByType(type: string): Promise<any[]> { const all = await this.mesuresMeteoRepository.find(); const mapValue = (record: MesuresMeteo, value: number, extra?: Record<string, any>) => ({ valeur: value, date: record.horodatage, ...(extra {}), }); switch (type) { case 'TemperatureAir': return all.map((m) => mapValue(m, m.temperature_air)); case 'TemperatureAirMoyenne': return all.map((m) => mapValue(m, m.temperature_air_moyenne)); case 'Hygrométrie': return all.map((m) => mapValue(m, m.hygrometrie)); case 'HygrométrieMoyenne': return all.map((m) => mapValue(m, m.hygrometrie_moyenne)); case 'Pluviometrie': return all.map((m) => mapValue(m, m.pluviometrie)); case 'Ensoleillement': return all.map((m) => mapValue(m, m.ensoleillement)); case 'EnsoleillementMoyenne': </pre>

	<pre> return all.map((m) => mapValue(m, m.enseillement_moyenne)); case 'DirectionVent': return all.map((m) => mapValue(m, m.vent_direction)); case 'VitesseVent': return all.map((m) => mapValue(m, m.vent_vitesse)); case 'VitesseVentMoyenne': return all.map((m) => mapValue(m, m.vent_vitesse_moyenne)); default: return []; } } } </pre>
Création d'un controller qui met à disposition les données en fonction de type	<pre> import { Controller, Get, Param } from '@nestjs/common'; import { MesuresMeteoService } from './mesures-meteo.service'; import { MesuresMeteo } from './mesures-meteo.entity'; @Controller('mesures-meteo') export class MesuresMeteoController { constructor(private readonly service: MesuresMeteoService) {} @Get('type/:type') async findByType(@Param('type') type: string) { return this.service.findByType(type); } } </pre>
Configuration du main.ts	<pre> import { NestFactory } from '@nestjs/core'; import { AppModule } from './app.module'; import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger'; async function bootstrap() { const app = await NestFactory.create(AppModule); const config = new DocumentBuilder() .setTitle('API Hippodrome') .setDescription('API pour la gestion des capteurs et des mesures') .setVersion('1.0') .build(); const document = SwaggerModule.createDocument(app, config); SwaggerModule.setup('api', app, document); } </pre>

	<pre>await app.listen(3000); } bootstrap();</pre>
--	---

L'ensemble de la structure de l'API a ainsi été conçu de manière modulaire, en respectant les bonnes pratiques de développement avec NestJS, ce qui facilite la maintenance et l'évolution du projet.

4.2 - Tests unitaires

Pour valider le bon fonctionnement de l'API REST, plusieurs tests unitaires ont été réalisés à chaque étape du développement. Ces tests ont permis de vérifier que chaque module (entity, service, controller) fonctionne comme attendu de manière isolée. Les tests ont été effectués principalement sur le module mesures_meteo, qui a servi de modèle pour les autres modules.

Nous allons maintenant détailler un exemple de test unitaire réalisé sur ce module.

4.2.1 -Test unitaire du module mesures_meteo

Élément testé :		Service mesure-meteo.service		
Objectif du test :		Vérifier que les routes HTTP fonctionnent comme prévu (POST, GET).		
Nom du testeur :		Nathan MAILLOT	Date :	30/04/2025
Moyens mis en œuvre :		Logiciel : Swagger	Matériel : Machine de développement	Outil de développement : Visual Studio Code
Procédure du test :				
Id	Description du vecteur de test	Résultat attendu	Résultat obtenu	Validation (O/N)
1	GET /capteurs pour récupérer toutes les mesures	Code 200 et tableau de mesures	[{ "id_capteur": 1, "hippodrome": { "id_hippodrome": 1, "nom": "Hippodrome de Paris", "pays": "France", "ville": "Paris", "superficie": 150.5, "nb_pistes": 3 }, "type_capteur": "météo", "nom_capteur": "Capteur Météo 1", "localisation": "Entrée nord" }, { "id_capteur": 2, "hippodrome": { "id_hippodrome": 1,	

			<pre> "nom": "Hippodrome de Paris", "pays": "France", "ville": "Paris", "superficie": 150.5, "nb_pistes": 3 }, "type_capteur": "terrain", "nom_capteur": "Capteur Sol 1", "localisation": "Zone centrale" }, { "id_capteur": 5, "hippodrome": { "id_hippodrome": 1, "nom": "Hippodrome de Paris", "pays": "France", "ville": "Paris", "superficie": 150.5, "nb_pistes": 3 }, "type_capteur": "meteo", "nom_capteur": "meteo-2", "localisation": "zone 2" }] </pre>	
2	GET /mesures-meteo/type avec un type inexistant	Code 404	[]	O
3	GET /mesures-meteo/type avec un type existant	Code 200 et tableau de mesures spécifiées	<pre> [{ "valeur": 18.2, "date": "2025-04-18T08:00:00.000Z" }, { "valeur": 19.3, "date": "2025-04-18T09:00:00.000Z" }, { "valeur": 21, </pre>	O

			<pre>"date": "2025-04-18T10:00:00.000Z", { "valeur": 22.4, "date": "2025-04-18T11:00:00.000Z" }]</pre>	
Conclusion du test :		Les tests unitaires effectués sur les routes HTTP de l'API confirment que la validation des entrées est correctement gérée, que les erreurs sont bien remontées, et que les données sont stockées comme prévu. Le module est donc validé pour un usage en environnement de test.		

4.2.2 - Problèmes rencontrés

Problème : La route GET renvoyait une liste vide malgré des données censées être présentes

Solution : Vérification de l'insertion effective des données et correction des erreurs de requêtes SQL dans le service

5 - Bilan de la réalisation personnelle

Points du projet validés

- Installation de ChirpStack v4 : L'installation de ChirpStack a été réalisée avec succès sur une machine virtuelle Ubuntu 22.04. La passerelle LoRaWAN a été correctement configurée pour transmettre les données vers le serveur ChirpStack.
- API REST : L'API REST a été développée et est opérationnelle. Elle permet de récupérer les données des capteurs et de les exposer sous forme de JSON.
- Schéma de base de données : Le schéma de la base de données avec les tables pour hippodrome, capteurs, mesures_meteo, et mesures_terrain a été créé et validé, avec une gestion correcte des relations entre les tables.
-

Parties restant à développer

- Améliorations de l'IHM : L'interface utilisateur de ChirpStack pourrait être enrichie pour offrir davantage de visualisations des données (graphes, cartes interactives) et une gestion plus fine des utilisateurs et des permissions.
- Tests sur le terrain : Il reste à effectuer des tests réels sur le terrain pour vérifier l'efficacité de la collecte de données et la réactivité du système face aux variations des conditions météorologiques.

Améliorations possibles

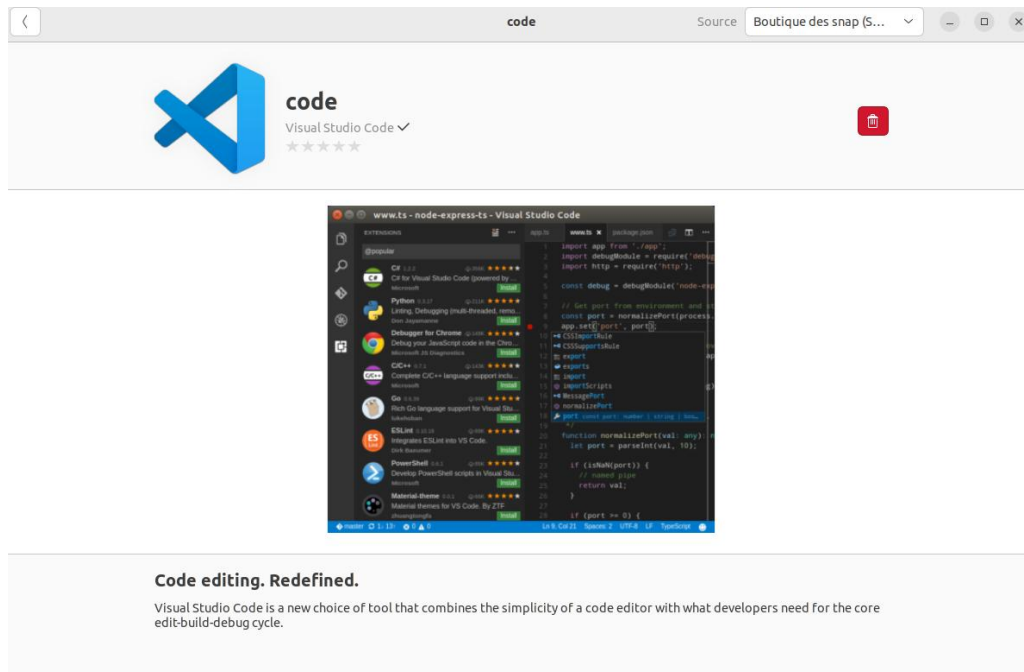
- Ajout de nouveaux types de capteurs : Il serait pertinent d'ajouter d'autres types de capteurs, par exemple pour mesurer la qualité de l'air ou d'autres paramètres liés à l'hippodrome.
- Sécurisation des communications : Mettre en place des protocoles de sécurité (SSL/TLS) pour chiffrer les échanges de données entre les capteurs, la passerelle et les serveurs.

Conclusion personnelle

Ce projet est une excellente occasion de développer mes compétences dans la gestion de projets IoT, l'intégration de systèmes complexes et la personnalisation d'outils open-source comme ChirpStack. Bien qu'il reste encore quelques aspects à perfectionner, je suis satisfait des progrès réalisés jusqu'ici et je suis confiant dans la réussite des prochaines étapes.

6 - Annexe

1.



2.

